

Feature: Gallery lightbox

From build-plan: feature 10

Branch: `feature/gallery-lightbox`

Goal

Item 4 promises the work "at a size where the quality is actually visible". A grid tile renders at **278 CSS px**, measured on the running page at 1440 wide, because the gallery sits inside a container capped at `--site: 1200px`. Tapping a photo has to open it large, or that promise is not kept.

In scope

- Tapping or keyboard-activating any gallery tile opens that photo large.
- Next and previous, by button, by arrow key, and by swipe.
- A thumbnail strip showing position, clickable to jump.
- Escape closes; focus returns to the tile that was opened.
- Navigation respects the active filter chip: with Nails pressed, next moves through nail photos only.
- Both locales. Every new string lands in `en.ts` and `es.ts` or the build fails.

Out of scope

- The service pages' `WorkStrip`. Same figures, but a second surface doubles the review; it can adopt the same component later.
- Preloading the neighbouring photo.
- Pinch zoom or pan inside the lightbox.
- Fixing `sizes` on the grid. It declares `25vw` where the element is 278px, so the browser fetches a larger file than it needs. Real, one line, and **its own fix** - do not smuggle it in here.
- An `onError` state for a photo that fails to load. The grid has the same gap today; fixing one surface and not the other is worse than fixing neither.

Build loop

`workflow.stepReview` is `feature`: work through the steps and present one review packet at the end. `workflow.checkpointCommits` is `disabled`, so make no commits. `/complete` creates the single feature commit.

Build steps

- ☒ **1. Put the Dialog primitive in place.** Add the shadcn dialog component. It is built on `@base-ui/react`, which this project already has, so it should cost no new package. **Hard stop:** if adding it would install any npm dependency, stop and ask. Installing without asking is forbidden by the workspace rules, and this feature's whole shape assumes it is free. *Done when* `components/ui/dialog.tsx` exists, `git diff package.json` shows no change, and `npm run build` passes.
- ☒ **2. Add the lightbox strings to both dictionaries.** Under `gallery` in `dictionaries/en.ts`, then the same keys in `es.ts`: a per-tile trigger label, close, next, previous, and a position phrase carrying the current index and the total. *Done when* `npm run build` passes, which is what proves both locales have every key - a missing Spanish string is a type error here by design.
- ☒ **3. Hand the photo data to the client and make each tile a trigger.** `app/[locale]/gallery/page.tsx` already resolves alt text and service names on the server; pass that same resolved list to `GalleryGrid` as a new `photos` prop, alongside the existing `children`. Each `GalleryFigure` renders its contents inside a `<button type="button">` carrying its index, and `GalleryGrid` listens on the grid container and reads the index from the event target. **Delegation rather than an `onClick` per figure is the point:** the figures are rendered on the server today, and the file says so. Giving each one a handler would make the whole grid a client component and undo that. *Done when* clicking or pressing Enter on tile N opens a dialog showing photo N, Escape closes it, and focus returns to that tile.
- ☒ **4. Next and previous, filtered.** Buttons plus Left and Right arrow keys. The set being traversed is the photos matching the pressed chip, not all nine, and it wraps at both ends. *Done when* with a service chip pressed, next moves only through that service and wraps from last to first.
- ☒ **5. Thumbnail strip.** A row of thumbnails under the main image, marking the current photo and scrolling it into view as navigation moves. Clicking one jumps to it. *Done when* the strip marks the current photo, clicking a thumbnail changes the main image, and arrow-key navigation moves the strip's mark.
- ☒ **6. Swipe.** Pointer events, hand-rolled. No `embla-carousel-react`. *Done when* Frank has tried it on his own phone and judged it no worse than Primo Painters'. **If it does not reach that bar, stop and say so** rather than shipping it - adding embla is then a separate decision with its own permission, not a silent fallback.
- ☒ **7. Loading state, then verify.** The large image is a bigger file than the tile and will not be instant on a phone. Show something deliberate while it arrives rather than an empty box. *Done when* `npm run build` and `npm run lint` pass, and the browser shows: the dialog opening from a tile, arrow keys moving, Escape closing, focus restored, both locales, and no console errors.

Files / areas

| Path | Change | |---|---| | `components/ui/dialog.tsx` | new, from shadcn | |
`components/gallery/GalleryLightbox.tsx` | new, client | | `components/gallery/GalleryGrid.tsx`
| new `photos` prop, delegated click, owns open state | | `components/gallery/GalleryFigure.tsx` |
contents wrapped in an indexed button | | `app/[locale]/gallery/page.tsx` | pass the resolved photo
list | | `dictionaries/en.ts` , `dictionaries/es.ts` | new strings under `gallery` |

Data / contracts

- The `photos` prop is resolved data, not records. Each entry carries the ImageKit path, the already-resolved alt string, width, height and `serviceId` . Alt text lives in the dictionaries and is resolved on the server today; moving that resolution into the client would pull both locales into the bundle, which item 2 explicitly avoided.
- Index is position in `photos` , and that array is `orderedGallery()` . The button's index and the array must come from one server render, or delegation opens the wrong photo. The children and the `photos` prop are built from the same `images` array in the same pass.
- `width / height` stay the real cropped pixels. They reserve layout in the grid; the lightbox reuses them for its own aspect ratio.
- The lightbox declares its own `sizes` . The grid's `25vw` is wrong for a near-full-screen image and would fetch a file far too small.

Testing

No test runner is configured - `package.json` has `dev` , `build` , `start` and `lint` only - and `verification.logicTests` is `when-configured` , so no unit tests are required. Do not install one inside this feature; `/tests` exists for that and it is a separate decision.

The logic worth testing is the filtered next/previous wrap. Exercise it in the browser against a pressed chip, both directions, at both ends.

`verification.uiEvidence` is `when-available` and a dev server can run here, so every done-when above that names a visible behaviour is checked in a browser and not inferred from a green build.

Notes for the AI

- A green build has hidden three real bugs on this project, each found only by running the page: an inverted honeypot, a canonical pointing at the wrong URL, and an audit that passed zero routes. A dialog is behaviour; build output proves nothing about it.
- The grid's server-rendered children are deliberate. `GalleryGrid` 's own comment explains that pressing a chip re-renders three buttons and a count, not the grid. Keep that true.
- Filtering is CSS, not React. The container carries `data-filter` and each figure hides itself with a `group-data-[filter=x]/grid:hidden` variant. The lightbox must derive its traversal set from the

same pressed-chip state rather than from what is visually hidden.

- **No new dependency without asking.** Step 1 stops if the Dialog is not free.
- `data/gallery.ts` and several files under `blueprint/` are CRLF in the working tree. A rewrite must preserve the line endings or the whole file churns.

Open questions

None blocking. Two choices are taken here rather than asked, both reversible and neither constraining later work:

1. **The service tag is not repeated in the lightbox.** It sits on the tile as a caption; at full size the photograph is the subject. 2. **Navigation wraps rather than stopping at the ends.** With nine photos, stopping means a dead button on arrival, which reads as broken.

Status: verified

Built across several sessions, then reviewed against a running page on 2026-09-22 and found to have one real defect, which was fixed before completion. See "The sizing defect found in review" below.

All seven build steps are checked. Step 6 (swipe) was held unchecked until Frank judged it on his own phone, which is what its done-when required; his verdict on 2026-09-22 was that it "swaps pretty good."

Verification actually performed

| Check | Result | |---|---| | `npm run build` | exit 0 | | `npm run lint` | exit 0 | | `git diff package.json` | no change, so the Dialog cost no dependency | | Lightbox opens from a tile, `/en` and `/es` | yes, correct photo for the tile's index | | Arrow keys move | yes, counter advanced 1 to 2 in both locales | | Escape closes, focus returns | yes, `document.activeElement` back on `data-photo-index="0"` | | Spanish strings | `Foto 1 de 9`, `Cerrar`, `Foto anterior`, `Siguiente foto`, translated alt | | Swipe on a real phone | Frank, 2026-09-22, accepted | | Photo fills width or height | all nine, at 412, 744, 1024, 1180, 1440, 1536, 1920, 2560 | | Overflow and dialog fit | no horizontal overflow, surface within the viewport at every width above | | Console errors on a fresh load | none from this component |

Widths were measured on the running dev server by comparing the painted photo box against the frame box, not by reading the `<img>` box. `object-contain` letterboxes inside its own element, so the element's rectangle hides the very thing being measured.

The sizing defect found in review

Frank reported that some photos opened small on a phone and a tablet while looking correct on a 27 inch monitor, and ranked all nine by how wrong they looked. That ranking turned out to be a ranking by master pixel count, which is what identified the cause.

The image carried `h-auto max-h-full w-auto max-w-full`. Those constraints only ever **shrink** a photograph to fit; they never grow one. So each photo drew at its own intrinsic size and filled the frame only when that size happened to exceed it. The large masters clamped and filled. The small ones sat marooned in the middle, touching neither edge.

Measured before the fix:

| Viewport | Frame | Photos not reaching any edge | |---|---|---| | 412 | 396x783 | 7, 8 | | 744 | 728x1015 | 2, 3, 7, 8 | | 1024 | 985x1226 | even photo 1, drawn 922x860 | | 1440 | height-bound | none, which is why desktop never showed it |

The fix is one line: the whole constraint set became `size-full`, so the box fills the frame and the `object-contain` that was already there fits the photograph to it.

The trap that made this hard to read is that `naturalWidth` on a responsive image is not the file's pixels. For a `srcset` with `w` descriptors the browser divides the served pixels by the chosen candidate's density, and that quotient is what `width: auto` resolves against. `pink-floral-art` is a 741x1500 master; behind the 828w candidate at a 412px viewport it reported `naturalWidth` 368. Fetching the identical URL with a bare `new Image()` in the same page reported 741. That contradiction is the tell.

It also looked like a breakpoint problem and is not. At 375 the browser picks the 750w candidate, density lands on exactly 2.0, the intrinsic width comes out 370 against a 359px frame, so it clamps and fills. At 412 it picks 828w and comes out 369 against 396, so it does not. Two widths with no breakpoint between them behaving oppositely is the signature. No breakpoint could have fixed it, and adding one was the first thing proposed.

Departures from the spec

Step 7 asked for a deliberate loading state and the feature ships without one. A `bg-shot` background on the image was built and removed: with `object-contain` the background paints the element's box rather than the photograph's footprint, so it rendered as visible letterbox panels beside every photo whose aspect did not match the frame. That was worse than the empty space it was meant to cover. After the sizing fix the element's box is the whole frame, so a background there would be more wrong, not less. A loading state still has a case; it needs to be a sibling behind the image rather than a background on it.

The arrows are not hidden on small screens. They eat roughly 90px of a 375px screen where swipe is the natural gesture. One line, deliberately left out of this feature.

Not verified

- **No automated tests.** No runner is configured and `verification.logicTests` is `when-configured`, so none were required. The filtered next and previous wrap was exercised in the browser, not by a test.
- **Sharpness on a real high-DPR screen.** The browser pane reports DPR 1 and scales the viewport, so it cannot judge this. Frank judged it on his own devices and accepted it. One measured note stands:

`pastel-french-tips` has a 960px master and draws at 1200 CSS px on a 2560 screen, a 1.25x upscale and the only photo anywhere in the sweep that goes past 1:1.

- **Screens above 2560 and below 375** were not measured.

Implementation walkthrough

`components/ui/dialog.tsx`

The shadcn dialog, which is built on `@base-ui/react`. That package was already a dependency, so the primitive cost nothing to add; step 1 carried a hard stop in case it did, and `git diff package.json` is empty.

The primitive's own overlay is 10% black. That is right for a settings dialog and far too light behind a photograph, so the dimming moved onto the lightbox's own surface instead of being configured on the overlay.

`components/gallery/GalleryLightbox.tsx`

The surface's class string is the part that looks arbitrary and is not. Three separate fights are recorded in it:

- **`translate-none` as well as `transform-none`**. Tailwind v4 compiles `-translate-x-1/2` to the standalone `translate` property, while `zoom-in-95` leaves a scale on `transform`. Clearing only `transform` left the surface at -712,-500 while reporting a computed `top: 0px`, which reads as impossible until you know they are two different properties.
- **Each side zeroed by name rather than with `inset-0`**. tailwind-merge treats `inset` and `top / left` as different groups, so the primitive's `top-1/2 left-1/2` survived alongside `inset-0` and won.
- **The photograph is absolutely positioned inside a `relative` parent.** `max-h-full` on a flex child does not clamp, because a percentage max-height resolves against a parent with a definite height and a flex item's height is not that. The tallest photograph came out 1037x1728 inside a 900px gap and simply overflowed.

Focus is moved by a callback ref on the close button, through the `render` prop. Base UI does not forward a plain `ref`, so `<DialogClose ref={x}>` leaves `x.current` null. The tell was a focus call that measurably did nothing while focusing the same node by hand stuck, which means nothing was stealing focus and the ref never arrived. `render` is what hands the primitive's props to an element we own.

And moving focus in means restoring it yourself. The primitive returns focus to whatever held it when the dialog opened; because the lightbox deliberately focuses its own close button, that element has unmounted by closing time and focus lands on `<body>`. `GalleryGrid` remembers the trigger and refocuses it, which is why Tab after closing resumes from the photograph rather than the top of the page.

Arrow keys bind to the document, not the dialog. Focus starts on the close button and can be tabbed anywhere inside, so binding to one element would make the arrows work in some places and not others. The listener only exists while the lightbox is open.

Swipe is hand-rolled on pointer events. Three rules, and the middle one is why it is not four lines: a drag that is mostly vertical is the visitor scrolling rather than swiping, so it is ignored rather than swallowed. The threshold is a distance rather than a velocity, because a slow deliberate drag should count and a twitch should not. `embla-carousel-react` was the named fallback if this felt cheap on a real phone; it did not, so no dependency was added.

The main image is keyed on its path. Without the key React mutates one element in place and the previous photograph stays on screen until the new file has downloaded, which reads as the control having done nothing.

`sizes` differs from the grid's on purpose. The grid declares `25vw`; the lightbox declares `90vw` and `100vw`, because reusing the grid's would fetch a thumbnail and stretch it.

`components/gallery/GalleryGrid.tsx`

Client, and it earns that with two pieces of state: which chip is pressed and which photo is open. The photographs themselves stay server-rendered and arrive as `children`.

The traversal set is derived from chip state, not from the DOM. Filtering is CSS: the container carries `data-filter` and each figure hides itself with a variant, so nothing in the DOM says which photographs are visible. Reading state instead is what stops next and previous walking into a photograph that is hidden on the page behind the dialog.

`components/gallery/GalleryFigure.tsx`

Each tile's contents are wrapped in a real `<button>` carrying its index, and the button has **no handler**. The grid listens once on the container and reads the index off the event target with `closest`, because the click lands on the `<img>`, never on the button. An `onClick` per figure would have turned the whole grid into client code and undone the reason the photographs are passed in as children.

The `index` prop is optional, and that is the contract with `WorkStrip`: the service pages render this same figure with no lightbox around it, so without an index the photograph is drawn exactly as before, with nothing focusable and no interaction the page cannot honour.

The service tag carries `pointer-events-none`, because it sits on top of the button and would otherwise make the most aimed-at part of the tile the one part that does not open the photograph.

`dictionaries/en.ts` and `es.ts`

Five strings under `gallery`. Alt text is resolved on the server and passed as plain data, because resolving it in the client would pull both locales into the bundle. The dictionary is keyed so a missing Spanish string is a type error, which is what makes `npm run build` a real check on translation completeness rather than a formality.